

Lighting the Storm

An apprentice keeper repairs a failing tower, lets two shipwrecked sailors in from the storm, and crosses a worsening sea for the one part that can't be fixed on site, before a passenger ship is lost on the rocks.

By **Paul** 25 September 2026 First-person · single-player · PC (Steam)

Bracken Isle Light · Fl(2) W 15s

PART 1

Executive Summary

You are the new apprentice to Ted, keeper of the lighthouse on Bracken Isle. You repair the building's failing systems under his direction, but the beacon keeps flickering. A rowboat crashes onto the island; you watch from the window, then open the gate so the survivors, Jeff and Daniel, can get inside. With Jeff, you cross the storm in an engine boat to fetch a replacement burner from an abandoned outpost and race back to fit it before a passenger ship reaches the rocks. The core skill is **steering a powered boat past rocks and hazards as the storm worsens**. No combat, no monster, no jump scares; folk-horror details stay unexplained.

Design pillars

1 · The light must return

Everything leads back to the beacon: what you repair, what guides you home, what decides the ship's fate.

Test: does it connect to getting the light back?

2 · The storm sets the clock

Pressure comes from the ship's lights, weather and visibility, never a timer, meter or enemy.

Test: can the player feel time running out without UI?

3 · Your hands, your eyes

All first-person and physical. Guidance comes from Ted and Jeff, not markers.

Test: is it happening to the player, or shown to them?

4 · Dread you can read

Sustained unease, but the player always knows what to do and why they failed.

Test: does it add unease without confusion?

Scope rule: one lighthouse, one stretch of sea, one abandoned outpost; no new major locations.

Cast

The Apprentice (player), characterised through action and a handwritten journal. **Ted**, the gruff keeper who directs repairs out loud. **Jeff**, a coarse older seaman and the player's companion on both crossings. **Daniel**, a younger sailor, shaken and glad to be alive, who stays at the lighthouse.

Scope

	Prototype	Full game
Length	About 10 minutes	About 30 minutes
Content	One repair, crash and gate, both crossings, burner, win and fail	All repairs, outpost exploration, optional discoveries
Quality	Greybox art, placeholder audio	Final art, audio, dialogue, controller support
Time	6 weeks, solo, AI-assisted	A further 5–6 months (estimate)
Proves	Hazards are readable; steering is enjoyable	Commercial release on Steam

Stretch goal: waves that push or roll the boat. The crossings work fully without them; they are added only after steering passes its week-2 test and are cut first if the schedule slips.

Game Mechanics

One worsening night on duty, played once through the main story.

Core loop

- 01

Repair the lighthouse. Ted names the broken system; the player finds and fixes it. The beacon still flickers.
- 02

Watch the crash, open the gate. Seen from the window; Jeff and Daniel walk in on their own.
- 03

Board with Jeff. The hidden ship deadline starts here.
- 04

Cross to the outpost, steering past rocks and debris to Jeff’s calls.
- 05

Pick up the burner at the outpost.
- 06

Cross back as the storm peaks: denser hazards, lower visibility.
- 07

Fit the burner in the lamp room as the ship closes in.
- 08

Win: the ship sails into port. **Fail:** it runs aground.

Lighthouse repairs

Interdependent systems (lamp and optic, rotation clockwork, fuel and fog signal, stormproofing) are fixed through short first-person puzzles with a fixed toolkit, not a crafting menu. **Ted’s directions** are the first assist; their detail scales with difficulty. A handwritten journal tracks what is broken, and some repairs open new areas. When everything else works but the beacon still flickers, Ted goes to the window, and that dead end becomes the inciting incident.

Sea crossing

The engine boat runs at **one fixed speed with one control: steering**. The challenge is placed hazards (rocks, reefs, buoys, debris) along a route that narrows and darkens as the storm grows; the sea moves for atmosphere only. **Jeff’s calls** are the second assist and read from the same hazard data as the world, so they never mislead. Aids: a compass, the beam when heading home, and silhouettes in low visibility. A collision takes on water, slows the boat or causes a brief capsized and recovery, all costing time. The burner can be knocked loose on the return but is never lost.

Tension and the deadline

A countdown the player never sees as a number starts on boarding. The ship’s approaching lights and the rising storm show how much time is left. It is the only timer: it runs through both crossings, the pickup and the final repair. Mistakes cost time; only missing the deadline fails the run.

Act 1 · Lighthouse	Act 2 · Outbound	Act 3 · Return	Climax · Burner
Calm, tutorialised, no deadline	Moderate hazards, deadline running	Severe: hardest skill test	Multi-step repair on the same deadline

Difficulty, accessibility and presentation

One difficulty preset sets how specific Ted and Jeff are and how intense hazards feel, without changing the story. Calls are subtitled with speaker names; controls are remappable. Everything is first-person, with no cutaways. Warm lamplight plays against blue-grey storm light, the beacon’s beam is the constant visual anchor, and sound is diegetic: wind, rain, machinery, a distant foghorn, no shock stingers.

AI Architecture

One developer works with eight AI agent roles, each owning one department. Each is described by what the player experiences as a result of its work.

AI is used during development only. The shipped game makes no AI calls: dialogue, storm and ship are authored and scripted, so there is no runtime API cost or dependency.

Agent	What the player experiences	What it builds	Human review gate
Boat / Physics	How the boat answers the stick, how clearly collisions register, how fast they recover	Fixed-speed controller, collisions, tuning data; stretch wave layer	Developer's week-2 go/no-go on feel
Level Layout	Where they can go and see: the tower climb, the crash window, the gate, the sea route	Greybox scenes, anchors, sightlines	Traversal and sightline checks
Gameplay / Systems	The story in the right order, the deadline on boarding, fair penalties, one clear ending	Phase controller, save state, repairs, deadline, win/fail	10-minute slice with no dead ends
Lighting	The beacon as goal and guide; darkness that deepens while hazards stay readable	Beacon sweep, visibility presets	Beacon and hazards legible every phase
Sound	A storm they can hear worsening; Jeff audible above it; no stingers	Layered storm, state-based mix	Jeff audible at peak storm
Asset	Objects they handle; unsettling symbols and shrines	Props and set dressing with provenance	Clean import; fits reference sheet
Dialogue	Ted's directions, Jeff's calls and journal objectives: how they know what to do without markers	Lines, subtitles, difficulty variants as data	Voice and lore approved before recording
QA	A night with no dead ends, both outcomes reachable	Scenario list, bug reports	Developer reproduces findings

How the agents work

- **Bounded briefs:** each task names its goal, inputs, owned files, interfaces and acceptance checks.
- **Human judgement:** the developer owns steering feel, tone, art fit and story, and reviews and merges every change.
- **Pillars bind every agent:** work that fails a pillar's test is returned. Unresolved story choices are escalated, never decided silently.
- **Sequence:** developer settles decisions → Boat/Physics steering spike (other content waits) → Layout and Systems slice → Lighting and Sound feedback → Dialogue and Asset content → QA and playtests.
- **Handoffs** return editable source, licences, test steps and known limits, and must run from a clean checkout.

Technical Strategy

Engine, build contract, schedule, token budget and constraints.

Engine: Godot 4 (proposed)

Factor	Godot 4	Unity	Unreal 5	Phaser
First-person 3D	Good	Very good	Excellent	No (2D)
AI can edit scenes	Yes (text)	Partly	No (Blueprints)	Yes
Iteration	Fast	Medium	Slow	Fastest
Licence	Free, MIT	Free below threshold	Royalty above threshold	Free, MIT

Godot’s main gap, no built-in ocean, is off the critical path with waves as a stretch goal. Unity is the alternative if the developer knows C#; Unreal shuts out AI help; Phaser can’t meet pillar 3.

Build contract

- Separately owned scenes (lighthouse, sea, outpost, player) and one game-state controller; content stored as data with stable IDs.
- Phases: REPAIRS → CRASH_AND_GATE → BOARDING → OUTBOUND → OUTPOST → RETURN → FINAL_REPAIR → WIN | FAIL
- Invariants: no deadline before boarding; Jeff is the only companion; the outcome resolves exactly once.
- Audio, UI and dialogue react to events and never change progression. Hazards, boat reaction and Jeff’s calls share one hazard forecast.

Six-week prototype schedule

Wk	Focus	Exit test
1	Setup; fixed-speed boat, one steering axis; greybox sea and hazards	Boat steers and collides reliably
2	Steering feel, hazard readability, collision recovery (Gate 0)	Go/no-go: steering is fun on its own
3	Outpost pickup, harder return, storm visibility	Both crossings playable
4	Repair, crash, gate, Jeff and his calls	Full 10-minute path playable
5	Hidden deadline, win and fail, placeholder audio	Both outcomes reachable
6	Playtests with 3–5 new players, tuning, tester build (Gate 1)	Prototype checks pass

Later gates: 2 · Narrative alpha (all repairs, discoveries, dialogue); 3 · Content and polish (final art, audio, accessibility, 30-minute pacing). Waves start only after Gate 0 passes, sit behind an on/off setting, and are cut first.

Token Budget, Constraints and Feasibility

Token budget

Unit: one agent task of about 40 turns at a 60K-token context, roughly 2.4M input tokens (about 90% cache reads) and 40K output. Mix: Opus 5.5 for implementation (70%), Sonnet 5 for routine work (20%), Haiku 4.5 for search and formatting (10%).

~\$500 Prototype · ~250 tasks · range \$250–\$1,400	~\$2,050 Full game incl. prototype · ~835 tasks · range \$1,000–\$5,000	~20M Input tokens a day in the prototype, mostly cached
--	--	--

Model	Input / MTok	Output / MTok	Cache read	Per task
Claude Opus 5.5	\$4.00	\$20.00	\$0.20	\$2.36
Claude Sonnet 5	\$2.00	\$10.00	\$0.20	\$1.40
Claude Haiku 4.5	\$1.00	\$5.00	\$0.10	\$0.70

Excludes other vendors’ AI art, audio and voice tools, paid engine assets and the Steam fee. Usage is re-measured after week 1.

API and plan constraints

- **Plan limits:** a Claude Team plan with rolling 5-hour and weekly limits and no overage, so hitting a limit pauses work. Prototype pace is estimated at 30–50% of the weekly limit.
- **Caching:** exact-prefix matching; stable docs and briefs keep most input as cheap cache reads.
- **Context:** 1M tokens on Opus 5.5 and Sonnet 5, 200K on Haiku 4.5; narrow briefs keep usage far below this.
- **Opus 5.5:** thinking is always on; spend is controlled with the effort setting.

Feasibility in 6 weeks

Target	Verdict	Why
10-min prototype	Feasible, with conditions	A playtest build, not a product. Needs steering to pass in week 2, frozen scope, placeholder assets, week 6 for playtests.
30-min full game	Not feasible	27–31 weeks solo (about 5 with stock assets). Gaps: 3D art and animation, QA, Steam launch prep.

Risks: boat feel (tested first at Gate 0), scope creep (held by the pillars) and usage limits (monitored weekly). **Key open decisions:** confirming Ted speaks live, deadline length and checkpoints, what follows the fail state, controller support in the prototype. **Beyond scope:** companion choice, Daniel as a companion, a hidden coda, Full Gale mode.